

NEXUS

Causal Intelligence Risk Report

Repository: <https://github.com/postgres/postgres>

Generated: 2026-05-17T06:31:22

Total DPRs: 15 | Nodes: 162 | Relationships: 202

36.8/100 Organizational Risk Score

1. Executive Summary

This report analyzes **15** architectural Decision Provenance Records from the PostgreSQL repository. The analysis reveals **8** decisions with critical blast radius, **4** with high assumption decay risk, and **4** actively decaying assumptions.

Latest monitoring run (2026-05-17T06:35:26): Scanned 5 commits, evaluated 6 DPRs, raised 2 new alerts.

2. Decision Provenance Records

ID	Title	Component	Blast	Decay	Deps
DPR-001	8KB Fixed Page Size	Storage	CRITICAL	MEDIUM	5
DPR-002	TOAST - The Oversized-Attribute Storage	Storage	HIGH	LOW	1
DPR-003	MVCC via Tuple Versioning in Heap	MVCC	CRITICAL	HIGH	5
DPR-004	32-bit Transaction ID with Wraparound	MVCC	CRITICAL	HIGH	3
DPR-005	HOT (Heap-Only Tuple) Updates	MVCC	HIGH	MEDIUM	1
DPR-006	Vacuum as Separate Maintenance Process	Storage	CRITICAL	HIGH	2
DPR-007	Write-Ahead Logging (WAL) for Durability	WAL	CRITICAL	MEDIUM	3
DPR-008	Autovacuum as Background Daemon	Autovacuum	CRITICAL	HIGH	0
DPR-009	Multi-Process Architecture	ProcessModel	CRITICAL	MEDIUM	2
DPR-010	Snapshot Isolation as Default	MVCC	HIGH	LOW	1
DPR-011	Shared Buffer Pool with Clock-Sweep	Storage	CRITICAL	MEDIUM	1
DPR-012	Checkpoint-Based Crash Recovery	WAL	HIGH	MEDIUM	1
DPR-013	Full-Page Writes for Torn Page Protectio	WAL	HIGH	LOW	0
DPR-014	Streaming Replication via WAL Shipping	Replication	HIGH	LOW	0
DPR-015	External Connection Pooling Required	ProcessModel	HIGH	LOW	0

3. Active Assumption Decay Alerts

DECAYING — DPR-003: MVCC via Tuple Versioning in Heap

Evidence: High-update workloads on cloud storage creating bloat faster than vacuum can clean

DECAYING — DPR-004: 32-bit Transaction ID with Wraparound

Evidence: High-throughput OLTP systems (100K+ TPS) can generate 2B transactions in ~6 hours

DECAYING — DPR-006: Vacuum as Separate Maintenance Process

Evidence: TB-scale tables taking 6-12 hours to vacuum, cloud storage IOPS limits making vacuum expensive

DECAYING — DPR-008: Autovacuum as Background Daemon

Evidence: 20% of 1TB table = 200GB of changes before vacuum triggers, causing massive bloat

Live Monitoring Results

DPR	Confidence	Holds?	Trend
DPR-006	0.10	No	The original decay evidence (TB-scale tables taking 6-12 hou
DPR-007	0.90	Yes	The commit 'Improve WAL segment recycling under heavy write
DPR-008	0.20	No	The original decay evidence already highlighted that the '20
DPR-009	0.90	Yes	The commit 'Improve parallel query worker allocation' indica
DPR-011	0.85	Yes	The original decay evidence pointed to challenges with large
DPR-012	0.85	Yes	The assumption concerns the performance and manageability of

4. Knowledge Concentration & SPOF Risk

Top Contributors by Bus Factor Risk

Name	DPRs	Components	Risk Score
Tom Lane	9	4	132.0
Heikki Linnakangas	4	3	42.0
Vadim Mikheev	4	2	28.0
Original PostgreSQL team	2	2	16.0
Andres Freund	2	2	16.0
Alvaro Herrera	2	2	16.0
Fujii Masao	2	2	12.0
Jan Wieck	1	1	3.0

Component SPOF Scores

Component	DPRs	Humans	SPOF Score
MVCC	4	5	6.3
Storage	4	5	6.0
Autovacuum	1	2	6.0
ProcessModel	2	3	3.5
WAL	3	5	3.3
Replication	1	2	1.5

5. Counterfactual Analysis

CF-001: What if PostgreSQL used 16KB pages from the start?

Verdict: WORSE (confidence: 90%) | Downstream impact: 5 DPRs

In this alternate timeline, PostgreSQL launched with 16KB pages. While users benefited from less frequent TOASTing and more efficient HOT updates, the system constantly struggled with higher memory consumption for the buffer pool and significantly increased WAL volume. Database administrators had to
New problems:

- Significantly increased memory pressure in the buffer pool: for the same memory allocation, the number of distinct cache
- Increased internal fragmentation within data pages and consequently within the buffer pool, wasting more memory for small
- Increased I/O amplification for random reads: reading 16KB from disk when only a small portion of the page is needed.

CF-002: What if PostgreSQL used undo logs like Oracle/MySQL InnoDB?

Verdict: TRADEOFF (confidence: 90%) | Downstream impact: 5 DPRs

In this alternate timeline, PostgreSQL's storage engine was designed with an undo log from its inception. Users never experienced the challenges of table bloat or the complexities of 'VACUUM' for space reclamation. Instead, the focus shifted to efficient undo log management and optimizing reads that
New problems:

- Managing the undo log: its storage, purging mechanism (when no active transactions need old versions), and recovery after
- Potential for undo log bloat if long-running transactions prevent timely purging of old undo records.
- Increased write amplification: Updates/deletes would involve writing to both the main data page (new version) and the un

CF-003: What if XID was always 64-bit, no wraparound risk?

Verdict: BETTER (confidence: 85%) | Downstream impact: 3 DPRs

In this alternate timeline, PostgreSQL developers would have chosen 64-bit transaction IDs from the outset, accepting a higher storage cost per tuple. This decision would have fundamentally simplified MVCC logic, eliminating the need for 'VACUUM FREEZE' and the complex 'autovacuum' mechanisms
design

New problems:

- Increased storage footprint: Each tuple would be 8 bytes larger (4 bytes for xmin, 4 for xmax). This would lead to large
- Increased memory consumption: Larger tuple headers would consume more shared buffer memory, potentially reducing cache e
- Potential performance degradation: While 64-bit comparisons are simpler, the increased data size could lead to more cach

CF-004: What if PostgreSQL used threads instead of forking processes?

Verdict: WORSE (confidence: 95%) | Downstream impact: 2 DPRs

In an alternate timeline, PostgreSQL adopted a thread-per-connection model from its inception. While initial connection overhead was lower and memory footprint per connection was reduced, the development team quickly grappled with pervasive concurrency bugs, deadlocks, and server-wide crashes due to

New problems:

- Massive increase in complexity for concurrency control, requiring pervasive locking and careful design for all shared da
- Significantly higher risk of memory corruption, race conditions, and deadlocks, leading to server instability and potent
- Debugging would be orders of magnitude harder due to non-deterministic race conditions, complex thread interactions, and

CF-005: What if WAL defaulted to async group commits?

Verdict: TRADEOFF (confidence: 90%) | Downstream impact: 3 DPRs

In an alternate timeline, PostgreSQL prioritized raw transaction throughput from its inception, defaulting to group commit for WAL and asynchronous replication. This led to a database known for its blazing speed under high concurrency, but users quickly learned to explicitly configure synchronous co

New problems:

- Increased potential for data loss on primary crash: transactions logically committed but not yet `fsync`d as part of a
- Increased potential for data loss on primary failover: transactions committed on the primary but not yet replicated to a
- Applications would need to be explicitly designed to handle potential data loss and eventual consistency, or to opt-in t

6. Recommendations

CRITICAL: 4 assumptions actively decaying. Immediate architectural review recommended for: DPR-003, DPR-004, DPR-006, DPR-008.

HIGH: Knowledge concentration risk. Top contributor **Tom Lane** is involved in 9 DPRs. Cross-training recommended.

MEDIUM: 8 DPRs have critical blast radius. Changes to these decisions cascade across the entire system: DPR-001, DPR-003, DPR-004, DPR-006, DPR-007, DPR-008, DPR-009, DPR-011.

Report generated by Nexus Causal Intelligence Engine at 2026-05-17 12:09:15